Introduction to the C Programming Language

CMPE311
Dr. Kidd

Acknowledgements

- This lecture was created by Dr. Ryan Robucci with modifications by myself.

Learning Objectives

At the end of this lecture, the student will be able to describe:

- The origins of C and C++
- Compare and contrast the purpose of C and C++
- The programmatic ways C differs from C++
- Write a program which
  - Declares, initializes and use variables of fundamental type
- Explain the differences between C and C++ data and functions
- Explain at least 5 differences between C vs C++ programming
- All the control statements, keyword/reserved words, declaration and initialization
- At least 3 indentation conventions

Outline

- What is the C-Language?
- System Development vs Application Development
- C-Language usage
- Identifiers
- C Fundamental Data Types
- Integral
- Miscellaneous
- Rest of Data Types
- Control Structures

What is the C Language?

What is C?

- Language that “bridges” concepts from high-level programming languages and hardware
  - Assembly = low-level
  - Python = Very high-level
    - Abstracts hardware almost completely
- C maintains control over much of the processor
  - Can suggest which variables are stored in registers
  - Can directly access memory and many registers
  - Don’t have to consider every clock cycle and data movement as in assembler

C is not C++

- No encapsulation
- No data and function protections (public /private /protected)
- No templates
- Non-unique signatures (no function or operator overloading)
- No namespaces
- No object structure, reuse, inheritance, etc
- No string, boolean or other modern first order data types
- No runtime error checking or interrupts
- Doesn’t facilitate the development of applications

But C++ Contains C

- C++ was a platform for researching advanced programming concepts
  - Object Oriented Programming (OOP)
  - Operator and function overloading
  - Etc.
- C++ was initially implemented as a C preprocessor
  - C++ code was translated to C code for compiling
- C++ has all the dangerous features that C has
  - E.g. “goto” statement
  - You are taught to avoid those dangers
- C code is not C++ code
  - Even though you can write it as nameHere.cpp

C History

- Began at Bell labs between 1969 and 1973
- Strong ties to the development of the UNIX operating system
  - C was developed to take advantage of byte- addressability where B could not
- First published in 1978
  - Called K&R C
    - Maximum early portability
    - A pseudo “standard” before C was standardized

Defacto Application Language

- C is a system development language, not an application development language
- Ill-suited for application development
  - Expert usage required (to not get into trouble)
  - Easy to introduce bugs
  - More difficult to program
  - Longer development times (i.e. $$)
  - Older (structured programming paradigm)
- But produced most portable code at the time
  - Though not standardized, more standard than most other languages
  - You used C if you wanted to produce an application that
    - Had the highest likelihood to work on another platform
    - Required minimal changes

C Derivative Languages

- The result is that most modern languages
  - Derivative syntactically from C, at least in part
  - Typically are written using C
  - Exploit the C standard libraries
  - Have facilities to link with code written in C
- The result is that C, though ill-suited as an application language, became the dominant application language from 1980 to 2000
- Subsequent languages attempted to address the failings of C as an application language
  - C++, Java, C#, etc
  - C and its direct decedents continue to play a significant role in application development

The C Standard

- First standardized in 1989 by American National Standards Institute (ANSI)
  - Usually referred to C89 or ANSI C
- Slightly modified in 1990
  - Usually C89 and C90 refer to essentially the same language
- ANSI adopted the ISO.IEC 1999 standard in 2000
  - Referred to as C99
- C standards committee adopted C11 in 2011
  - Referred to as C11, and is the current standard
  - Many still developed for C99 for compatibility
  - C17 is essentially the same but with some fixes
  - C23 is the newest with significant additions

Usage Lags Standards

- C89 or C99 used mostly
  - MS standardized on C89 with some C99 extensions
- Legacy code requires use of older standards
  - If a company has 100,000 lines of code in C89, it doesn't make sense to rewrite it in C11
    - Why does this not make sense?
- New companies often use older standards
  - Integrate with older code
  - Programmers more familiar with older standards

System Development vs Application Development

C is a Dangerous Language

- C can be dangerous
  - No runtime error checks or interrupts
  - No heap memory management
  - Very limited data and function protections
  - Non-unique signatures
  - Access to very low-level and hardware structures
    - Can directly access and change registers, interrupts, etc
- C is more expensive to develop with
  - Programmers need more training ($$)
  - Programs need more debugging ($$)
  - More post-release bugs ($$)
  - More technical maintenance ($$)

System Development and Dangers

- Claim: C is dangerous because it is a system development language, not an application language.
  - Why does a system development language have to be dangerous?
  - How does systems development programming differ from applications development?

Systems vs Application Development

- Application Development
  - Labor cost dominates
  - Data structures largely the same
  - Functions largely the same
  - Skill required: Capable
  - Performance: secondary importance
  - Solutions: relatively simple
- Systems Development
  - Labor cost is secondary or tertiary factor
  - Data structures variable
  - Functions often unique
  - Skill required: Expert
  - Performance: Primary importance
  - Solutions: often very complex

Systems vs Application Development

Issue	Application	Systems
Labor	Major factor	Minor factor
Robustness	Major factor	Critical factor
Debugging	OK	Difficult
Solution space	broad	well-defined
Flexibility	Minor factor	Major factor
Lines of code	Relatively small	Typically huge
Maintenance	So-so	Major factor

All this translates to very expensive to develop and maintain

Difference: System vs App Development

- System development requires access to hardware and memory
  - Specialize register usage
  - Write direct interfacing routines like ISRs
  - Direct memory access for special HW tables
  - Unconventional data structure design and access
    - Efficiency and performance over 'safe'
- Conditions have changed since the development of 'C'
  - Processors ~6 million times larger (transistor # in processor)
  - Processors are ~100 million times faster (Inst per second)
- These changes shift development priorities
  - Performance less important
  - Minimizing debugging time more important

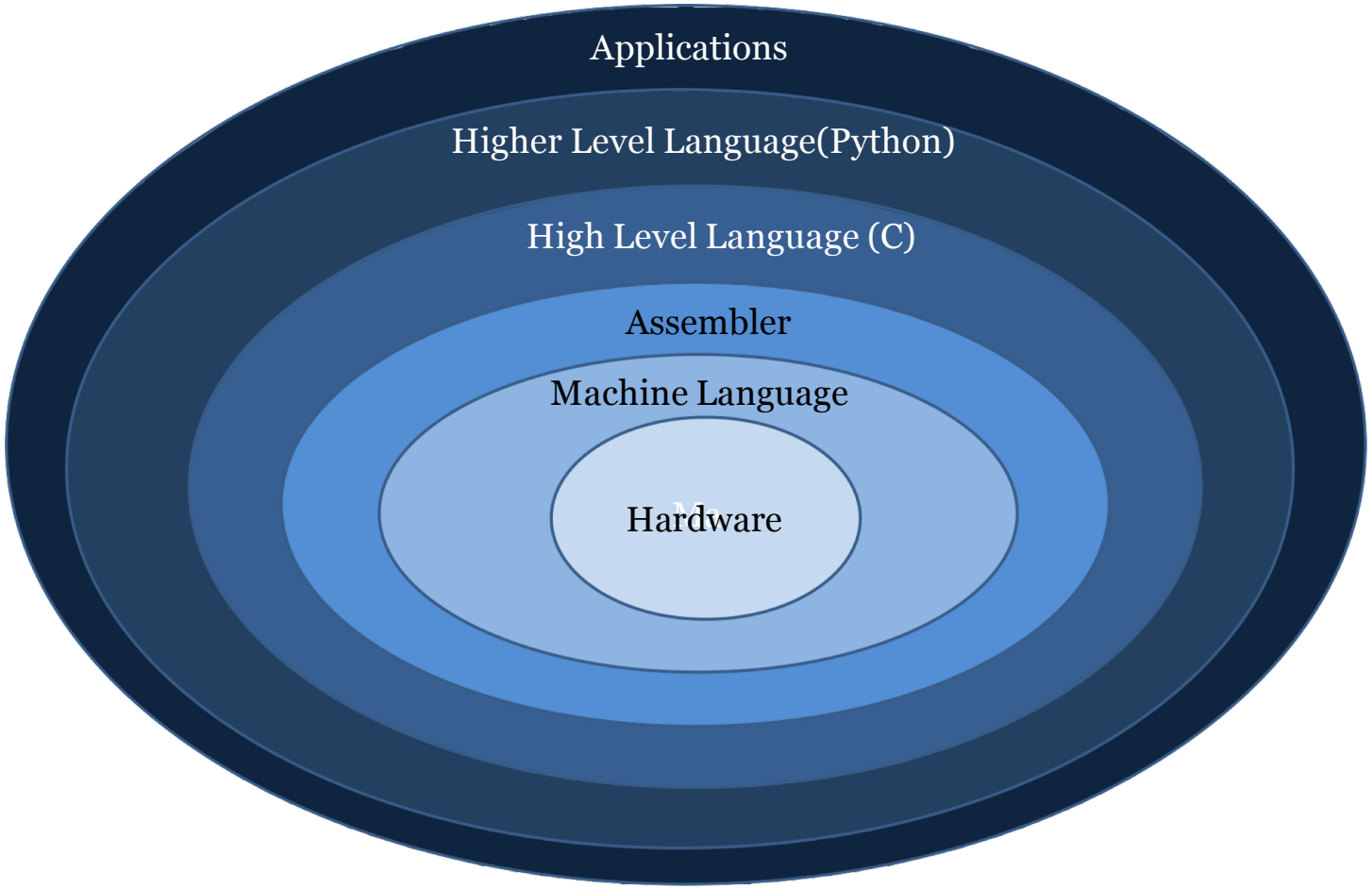
Why C and C++?

- Why did C become so popular?
  - Not standardized
  - Not suited for application development
  - Difficult to use
- Most portable language
  - Had the greatest chance of compiling everywhere
    - More that “standardized” languages
  - Thus the goto (sic) language for application development

C Resources

- http://cslibrary.stanford.edu/101/EssentialC.pdf
- http://publications.gbdirect.co.uk/c_book/
- MIT Open Courseware
  - http://ocw.mit.edu/courses/#electrical-engineering-and-computer-science

Hardware to Application Onion Model



C Language Usage

Hello World

```
/*
file header block comment
*/
#include <stdio.h>
int main( )
{
   // print the greeting
   cout << “Hello World\n”;
   return 0;
}
```

- Recognize this?
- This is a C program and not a “C++” program.
  - Even though cout is technically an object
- A C++ program uses C++ programming paradigms and principles
  - Encapsulation
  - Objects
  - Polymorphism
  - etc

Hello World

```
#include …
public class Greeting {
public:
   Greeting(String message)
      {…}
  int display(){…}
}

void main(…)
{
   Greeting msg( “Hello” );
   msg.display();
   return 0
}
```

- A ‘real’ C++ simple program
  - Use OOP principles
  - Defines a ‘Greeting’ class
  - Constructor setting greeting message
  - Member function to display greeting
- Too complex for “Hello World” so use ‘C’ program instead

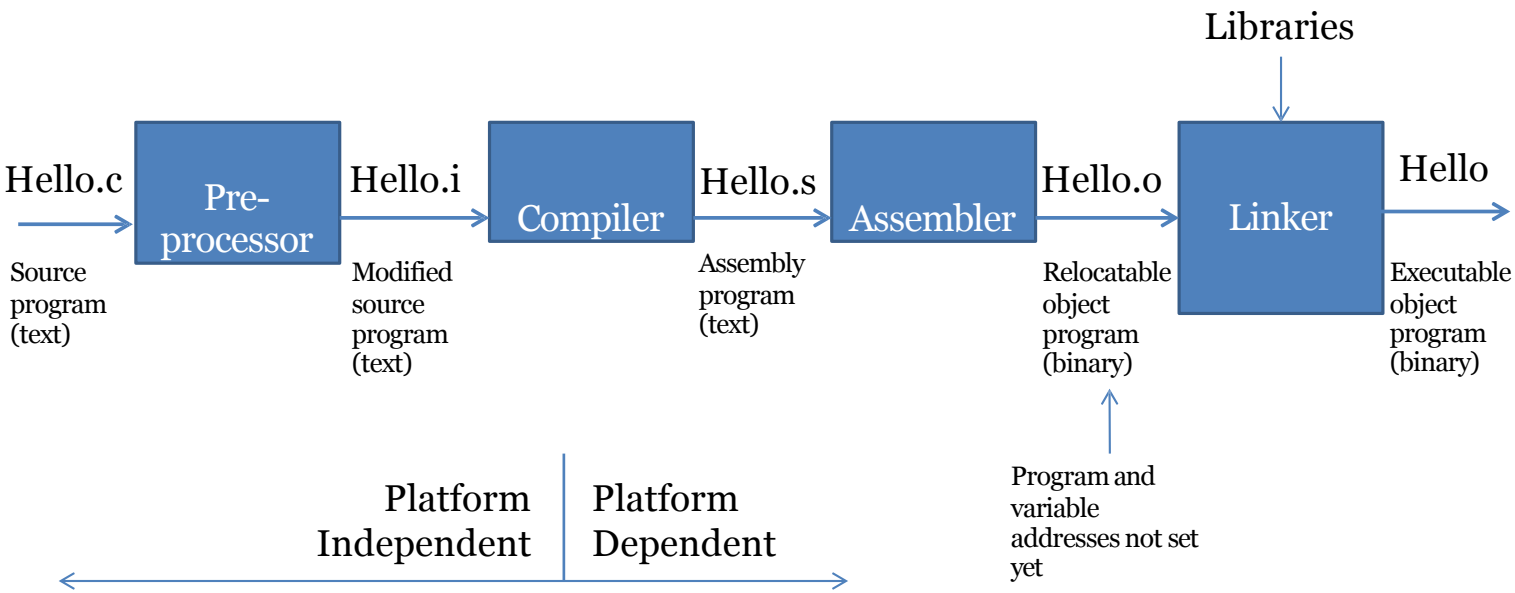
Compiling on Unix

- Traditionally the name of the C compiler that comes with Unix is “cc”
  - UMBC GL systems use the “GNU Compiler Collection”
    - “gcc” to compile C (and C++ programs)
  - Default name of executable program created by gcc is a.out
    - Can specify executable using “-o” operand

Compiler Options

- -c
  - Compile only, don't link
    - Create a .o file, but no executable
  - E.g. gcc –c hello.c
- -o fname
  - Name the executable filename instead of a.out
  - E.g. gcc –o hello hello.c
- -ansi
  - Enforce ANSI C standard, disable C99 features
  - ANSI C was the defacto most likely to work with any system
  - C99 has taken over that title
- -S
  - Compile to assembly (GAS) and stop
- -s
  - Compile to executable but do not delete assembly
- -O0
  - Do not apply any optimizations
- -g
  - Preserve symbol table and other debug information
- -Wall
  - Report all warnings

Compilation Flow



Program is executed by calling name of executable at Unix prompt:
E.g. unix>hello

Libraries

- Library is composed of predefined functions
  - As opposed to classes for OOP language
  - Examples include:
    - Char/String operations (strcpy, strcmp)
    - Math functions (floor, ceil, sin)
    - Input/Output Functions (printf, scanf)
- C/Unix manual – “man” command
  - Description of C library functions and unix commands
    - e.g. “man printf” or “man dir”

Compiler Vocabulary

- Preprocessor
  - Prepares file for compiler, handles processing macros, source selection, preprocessor directives, and file includes
- Compiler
  - Converts (nearly) machine independent C code to machine dependent assembly code
- Assembler
  - Converts assembly language to machine language of an object relocatable file (addresses not all resolved)
- Linker
  - Combines all object files and resolves addressing issues
- Loader
  - When executed, loads executable into memory
- Cross compiler
  - Compiler that runs on one platform but outputs code for another target machine (e.g. AVR is compiled on Intel)

Identifiers

Identifiers

- Identifier – name of a function or variable
- ANSI/ISO C standard
  - CASE SENSITIVE
  - First character must be alpha or _
  - May NOT be a C keyword such as int, return, etc…
  - No length limit imposed by standard
    - May have compiler limitation
- Good coding practices
  - Choose convention for capitalization of variables and functions
  - Symbolic constants should be all caps
  - Choose descriptive names over short names

Choosing Identifiers example

- T1, Temp1, Temperature1
- Which of the three above is most useful?
- Treat identifiers as documentation
  - Something which you would understand 3 years later
  - Don’t be lazy with naming, put effort into documentation
- C and C++ identifier rules essentially identical
  - C++ can use UTF (foreign characters) in comments
  - New reserved keywords

Declaring, Defining, Initialization

- C allows you to declare and define variables
- A declaration puts the variables name in the namespace
  - No memory is allocated
  - Sets identifier (name) and type
- A definition allocates memory
  - Amount depends on variable type
- An initialization (optional) sets initial value to be stored in variable

C Declaration Example

- In C, combined declaration and definition is typical

```
char example1; //definition and declaration
int example2 = 5; //def. decl. and init.
void example3(void){//def. and decl. of a function
    int x = 7;
}
```

- The "extern" keyword may be added to declare that definition will be provided elsewhere

```
extern char example1;
extern int example2;
void example3(void);
```

A function which does not provide definition is sufficient for the compiler. This declaration is called a prototype

Assignments

- Assignments set values to variables
- Uses equal “=“ character and end with semicolon
  - E.g. temperature1 = 3;
  - temperature2 = temperature1;

C Fundamental Data Types

Types

- Intrinsic (fundamental, built-in) types
  - Integral types
    - E.g. int, char, long
  - Floating-Point types
    - E.g. float, double
- Type synonyms (aliases) using “Typedef”
  - Keyword “typedef” can be used to give new name to existing type
  - Example:
    - typedef unsigned int my_type;
    - my_type a=1;
  - Useful for Structures (covered later)

Initialization

- Refers to the first assignment whether in declaration or afterward
- No constructors and only simple data types
- Until initialization, variables are considered uninitialized
  - Contents are unknown/unspecified/garbage
  - Exception: All objects with static storage duration (variables declared with static keyword and global variables) are zero initialized unless they have user-supplied initialization value
    - Still good practice to provide explicit initialization
- Initialization is not “free” in terms of run time or program space
  - Equivalent to a LDI

Integral

C Fundamental Data Types

Integral Data Types

- C data types for storing integers are:
  - int (basic integer data type)
  - short int (typically abbreviated as short)
  - long int (typically abbreviated as long)
  - long long int (C99)
  - char (C does not have “byte”)
  - int should be used unless there is a good reason to use one of the others
- Number of bytes
  - char is stored in 1 byte
  - Number of bytes used by other types depends on machine being used

Integral Type Sizes

- C standard is specifically vague regarding size
  - A short must not be larger than an int
  - An int must not be larger than a long int
  - A short int must be at least 16 bits
  - An int must be at least 16 bits
  - A long int must be at least 32 bits
  - A long long int must be at least 64 bits
- Check compiler documentation for specific lengths

Integral Specifiers

- Each of the integral types may be specified as:
  - Signed (positive, negative, or zero)
  - Unsigned (positive or zero only) (allows larger numbers)
- Signed is default qualifier
- Be sure to pay attention to signed vs. unsigned representations when transferring data between systems. Don’t assume.

Common Embedded User Types

- To avoid ambiguity of variable sizes on embedded systems, named types that make size apparent should be used
- WinAVR has predefined custom types:
  - int8_t - signed char
  - uint8_t - unsigned char
  - int16_t - signed int
  - uint16_t - unsigned int
  - int32_t - signed long
  - uint32_t - unsigned long
- These are defined in inttypes.h using typedef command

Floating Point Types

- C data types for storing floating point values are
  - float – smallest floating point type
  - double – larger type with larger range of values
  - long double – even larger type
- double is typically used for all floating point values unless compelling need to use one of the others
- Floating point values may store integer values

Floating Point Type

- C standard is again unspecific on relative sizes
  - Requires float < double < long double

- Valid floating point declarations:

```
float avg = 10.6;
double median = 88.54; double homeCost = 10000;
```

Character Data Types

- C has just one data type for storing characters
  - char – just 1 byte
  - Because only 1 byte, C only supports ASCII character set
- Example assignments:
  - char x = ‘A’;
    - Equivalent to : char x = 65;
    - ASCII character set recognizes ‘A’ as 65

Embedded Note: Floating point operations

- Most embedded processors do not have floating point hardware
  - There's no need in, say, your microwave or thermostat
- C programs still have floating point support
  - Just implemented as software

Boolean Data Type

- ANSI has no Boolean type
- C99 standard supports boolean data type (bool)
  - Many companies still don't use it to maintain compatibility with C89
- In C99, to use bool, true, and false you must include stdbool.h

```
#include <stdbool.h> bool
isRaining = false;
if(isRaining)
    printf("Bring your umbrella\n");
```

Enumeration Constants

- C provides the *enum* as a list of named constant integer values (starting at 0 by default)
- Behaves like integers
- Names in *enum* must be distinct
- Often a better alternative to #defines
  - Specifically defining symbols to values and not very general macros
- Example

```
enum months{ JAN=1, FEB, MAR, APR, MAY, JUN, JUL, AUG,
      SEP, OCT, NOV, DEC};
enum months thisMonth;
thisMonth = SEP;  //ok
thisMonth = 42;    //unfortunately, also ok
```

enum syntax

- From: gnu.org c reference manual
  - enum emotion_state { neutral, happy, sad, worried, calm, nervous };
    - Default assignment: neutral=0; happy=1, sad=2, worried=3, …
  - enum emotion_state { neutral = 2, happy = 5, sad = 20, worried = 10, calm = -5, nervous = -300 };

Is void A Datatype?

- The answer is yes and no
  - The specification considers it a data type
  - But it's generally used as a placeholder
- Use#1: A placeholder that means "Not" or "None"
  - void function(); //doesn't produce a value
  - int function(void); //no value is passed
- Use#2: An untyped pointer
  - Think of it as the parent class of all pointers
  - None really useful until it is cast

- Here's a good reference: https://stackoverflow.com/questions/3487689/is-void-a-data-type-in-c

COVERED 311 Tue 2/10/26

- COVERED:
- NEXT:
- READING:
- HOMEWORK:

Other Fundamental Datatypes

- The usual
  - float, double
- In C99 and C11
  - bool
- These have limited use in embedded programming
  - Limited memory, low frequency, limited word length
  - Many uControllers don’t have floating point operations

- Custom Data Types using ‘typedef’
  - typedef int Integer;
  - typedef struct { /* members */} MyTypeName;

Const qualifier

- Any of the variable types may be qualified as *const*
- *const* variables may not be modified by your code
  - Any attempt to do so will result in compiler error
    - Compile time checking
    - No runtime checks
  - Must be initialized when declared
    - E.g. const double PI = 3.14159;
    - const int myAge = 24;
    - const float PI; //valid, PI=0
    - PI = 3.14159; //invalid

Variable Declaration

- ANSI C requires all variables be declared at the beginning of the “block” in which they are defined
  - Before any executable line of code
- C99 allows variables to be declared anywhere in code
  - Like java and C++
- Regardless, variables must be declared before they can be used

Arrays

Arrays

- C supports arrays as a basic data structure
- Indexing starts with 0
  - myArray[0]
- ANSI C requires size of array be a constant
- Declaring and initializing arrays:
  - int grades[30];
  - int areas[10] = {1,2,3};
  - long widths[12] = {0};
  - int IQs[] = {120, 121, 99, 154};

Variable Size Arrays

- C99 allows size of array to be a variable
  - int numStudents = 30;
  - int grades[numStudents];

Multi-Dimensional Arrays

- C supports multi-dimensional array
- Subscripting provided for each dimension
- For 2-d arrays:
  - First dimension is number of “rows”
  - Second is number of “columns” in each row

  - int board[4][5]; // 4 rows, 5 columns
  - int x = board[0][0]; //1st row, 1st column
  - int y = board[3][4];//4th (last) row, 5th (last) column

sizeof()

- Because sizes of data types in C standard are vaguely specified, C provides sizeof() operator to determine size of any data type

- sizeof() should be used everywhere the size of a data type is required
  - Maintain portability between systems
  - Remember, fundamental types can vary between systems and different C compilers
    - Definition of int: ”int must not be larger than a long int”

Aside: C for Science and Engineering

- C ill suited for a lot of science and engineering
  - Especially “big” science and engineering
- C arrays are inherently inefficient for performing matrix operations.
  - C organizes data as column major order ill suited for matrix operations
- Why? C data organization is organized for setting up systems related data structures
  - column-major and linked lists
- C++ has inherited this same organization
  - And has the same issues with regards to science and engineering
- C and C++ still used for science and engineering despite the inefficiencies

- Side note: FORTRAN is still one of the fastest super computing languages
  - developed between 1953-1957

Operators

Arithmetic Operators

- Arithmetic operators are the same as java
  - = : assignment
  - +,- : plus, minus
  - *,/,% : multiply, divide, modulus
  - ++, --: increment, decrement (pre and post)
- Combinations are the same
  - +=, -= : Plus equal, minus equal
  - *=, /=, %=: multiply equal, divide equal, mod equal

Logical Operators

- Logical Operators are closely similar in C and python and result in boolean value
  - &&: and
  - || : or
  - ==, !=: equal and not equal
  - <, <=: less than, less than or equal
  - >, >=: greater than, greater than or equal
- Integral types may also be treated as boolean expressions
  - 0 considered false
  - Any non-zero is considered true
  - For compatibility with pre-C99

Control Flow Statements

Same as C++

- if-then-else
- switch-case
- for, while, do-while
- break, continue, return
- goto
  - Exists but not taught
  - Seldom used because dangerous

Different from C++

- No signal/interrupt handling
- No try, catch or throw exception control

Control Structures

Control Structures

- Both languages support the following control structures
  - For loop
  - While loop
  - Do-while loop
  - Switch statements
  - if and if-else statements
  - Braces ({,}) are used to begin and end blocks

Curly Braces

- Used to group multiple statements together
  - Control blocks, functions
  - Statements execute in order

```
int main(){
    int i=7;
    if(i==7) {
        i=i+j;
        int k; //forbidden by c89 standard (c99 okay)
        k=i*I; //variables declared at top of block
    }
}
```

If - Else block

```
if (expression) <statement>
    e.g. if(x>3) x+=1;      //simple form
if(expression) {    //simple form with {} to group
    <statement>;
    <statement>;
}
if(expression){//full if/else form
      <statement>;
}  else  {
      <statemen>;
}
```

If - Else If – Else block

```
if(expression1) {
    statement 1;
} else if (expression2) {
    statement2;
} else {
    statement3;
}
```

Spacing Variation (Be Consistent)

//K&R
 	
if(expression) {
  statement;
}else{
  statement;
}
	//GNU
	
	if (expression)
	  {
	  statement;
	  }
	  else
	  {
	  statement;
	  }
		//Allman
		
		if (expression)
		{
		  statement;
		}
		else
		{
		  statement;
		}

There are many spacing styles for logic blocks (>9). Pick one and **be consistent**.

Switch

```
switch (expression) {
   case const-expression-1:
      statement;
      break;
   case const-expression-2:
      statement;
      break;
case < const-expression-3>: //combined
case 3 and 4 case <const-expression-4>:
      statement;
      break;
case < const-expression-5>: /no break
      mistake? maybe statement;
case <const-expression-6>:
      statement;
      break;
default:/
      optional
      statement;
}
```

Omitting the break statements is a common error --it compiles, but leads to inadvertent fall-through behavior. This behavior is just like the assembly jump tables it implements.

Note: Switch, Unintentional Bug

```
switch (expression) {
   case const-expression-1:
      statement;
      break;
   case const-expression-2:
      statement;
      break;
   case <const-expression-3>: //combined case 3 and 4
   case <const-expression-4>:
      statement;
      break;
   case <const-expression-5>: //no break mistake? maybe
      statement;
   case <const-expression-6>:
      statement;
      break;
   default: // optional
      statement;
}
```

<= must be executable, i.e. generate code. This only applies to the first / top-most case.

While – Do While

```
while(expression){      //executes 0 or more times
   statement;
}

do{                     //executes 1 or more times
   statement;
}while(expression)
```

For loops

```
for(initialization; continuation; action){ statement;
}
for(; continuation; action){ statement;
}
```

- Initialization, continuation and action are all optional.
- May optionally declare a variable in initialization (C99 standard)
- Continuation condition must be satisfied for every execution of statement, including the first iteration
- Action is code performed after the statement is executed

For loops

```
int i = 99;
for(; i!=0;){
        statement;
        i-=1;
}

for (int i = 99; i!=0; i=i-1){
        statement;
}
```

- These are equivalent statements.
- The second one is much more readable.
- The second one also uses the C99 variable declaration inside the for loop. This may not work on AVR.

Break

```
while(expression){
   statement;
   statement;
   if(condition)
      break;
   statement;
   statement;
}
//control jumps here on break.
```

Continue

```
while(expression){
  statement;
  if(condition)
    continue;
  statement;
  statement;
  //control jumps here on continue
}
```

Continue vs Break

- Continue skips the rest of the current iteration
- Break exits the loop entirely

- for(i=0;i<5;i++){ printf("%d\n",i); break }
  - Breaks out of the entire for statement when i=0
  - Never completes rest of loops (i=1,2,3,4)
- for(i=0;i<5;i++){ if (i%2) continue; printf("%d\n",i); }
  - only exits odd loops of i

Conditional Expression

- Also called the Ternary Operatory

- ?: (tri-nary “hook colon”)
  - C: int larger=(x>y ? x : y);
  - Python: larger=x if x>y else y

- Syntax: expression1 ? expression2:expression3
  - if expression1 is true, return expression2, else return expression3
  - Use this sparingly since it makes code less readable

Other Operators

- These operators are very similar in C and Java

- <<,>>,&,~,|,^ ;bit operators
- <<=,>>=,&=,~=,|=,^= ;bit equal operators
- [] ;brackets (for arrays)
- () ;parenthesis for functions and type casting

- ^ ;binary XOR

#defines

- The #define directive can be used to give names to important constants
  - Makes your code more readable and changeable
- The compiler's preprocessor replaces all instances of the #define *name* with the text it represents
  - Replaces *name* with text only, no interpretation of syntax
  - Note: no terminating semi-colon

```
#define PI 3.14159
…
double area = PI * radius * radius;
```

#define vs. const

- #define
  - Pro – no memory is used for the constant
  - Pro – useful for more than just numerical/char/etc constants
  - Con – cannot be seen when code is compiled
    - Removed by pre-compiler
  - Con – not real values, have no type
  - Con – can cause difficult to identify compiler errors
- const variables
  - Pro – real variables with a type
  - Pro – Can be examined by debugger
  - Con – take up memory
  - Con – limited to assignment and usage in expressions

Examples

```
const int NUMBER = -42;
int main(){
   int x = -NUMBER;
}
```

If replaced with a # define, will generate compiler error
(-- is a decrement operator)

```
#define NUMBER 5+2
int x = 3 * NUMBER;
```

Value of x is 17 with #define
Wanted 21
(int x = 3 * 5 + 2) vs (int x = 3 * 7)

```
#define NUMBER 5+2;
int x =  NUMBER * 3;
```

Compiler error
int x = 5 + 2; * 3;

C Functions

- C Functions (no explicit procedures)
  - Have a name
  - Have a return type
  - May have parameters
  - Procedures
    - Other languages separate ‘functions’ from ‘procedures’ where C doesn’t
    - A procedure is a function returning void
- Before a function may be called, its “prototype” must be known to the compiler
  - Verify that function is being called correctly
  - Accomplished by:
    - Providing entire function prior to calling function in code
    - Provide function prototype prior to calling in code and providing function elsewhere

C Functions

- Unlike Java, a function is C is uniquely identified by its name
  - No concept of method overloading
  - No mangling to create a unique signature
  - There can only be one main() function in a C application
- UMBC coding standards dictate function names begin with UPPERCASE letter
  - E.g. AddThreeNumbers() instead of addThreeNumbers

Simple C Program

```
#include <stdio.h>
typedef double Radius;
#define PI  3.1415
/* given the radius, calculates the area of a circle */
double CircleArea( Radius radius ){
   return ( PI * radius * radius );
}
// given the radius, calculates the circumference of a circle
double Circumference( Radius radius ){
   return (2 * PI * radius );
}
int main( ){
   Radius radius = 4.5;
   double area = circleArea( radius );
double circumference = Circumference( radius );  // print the results return 0;
}
```

Typical C Program

Includes

Defines, typedefs, data type definitions, global variable declarations, function prototypes

Main

Function Definitions

```c
#include <stdio.h>

typedef double Radius;
#define PI 3.1415

/* function prototypes */
double CircleArea( Radius radius );
double Circumference( Radius radius );

int main( )
{
   Radius radius = 4.5;
   double area = circleArea( radius );
   double circumference = Circumference( radius );

   // print the results
   return 0;
}

/* given the radius, calculates the area of a circle */
double CircleArea( Radius radius )
{
   return ( PI * radius * radius );
}

// given the radius, calcs the circumference of a circle
double Circumference( Radius radius )
{
   return (2 * PI * radius );
}
```